

CODE Bras Robotisé - Lotfi Chenoun

/*****

* Projet : Bras robotisé

* Auteur : Lotfi Chenoun 6A

* Date de dernière modification : 06/02/2026

*

* Objectif :

* Contrôle d'un bras robotisé à 4 servomoteurs (base, épaule,

* coude, pince) à l'aide d'une manette Bluetooth (mode PS5)

* ou de deux joysticks analogiques (mode Analogique).

* La pince est protégée par une mesure de courant via un

* capteur ACS712 afin d'éviter les surcharges mécaniques.

*

* Librairie de la manette écrite par : F. Kapita

*

* Mode Analogique - Brochage joysticks (schéma COM-09032) :

* Joystick 1 : AX1 → GPIO25, AY1 → GPIO26, SEL1 → GPIO13

* Joystick 2 : AX2 → GPIO34, AY2 → GPIO35, SEL2 → GPIO39

*

* Mapping mode Analogique :

* JS1 axe X → Base

* JS1 axe Y → Épaule

* JS2 axe X → Pince

* JS2 axe Y → Coude

*****/

#include <M5Unified.h>

#include <Wire.h>

#include <Adafruit_PWMServoDriver.h>

#include "controller.h"

```

/* -----
   Configuration du driver PWM (PCA9685)
   ----- */

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);

#define SERVOMIN 100
#define SERVOMAX 610

/* Indices des servomoteurs */
#define SERVO_BASE    0
#define SERVO_EPAULE  4
#define SERVO_COUDE   8
#define SERVO_PINCE   12

/* -----
   Capteur de courant ACS712 (pince)
   ----- */

#define PIN_COURANT_PINCE 36
#define SENSIBILITE_ACS712 0.1 // 20A = 0.1 V/A
#define COURANT_MAX_PINCE 1.5 // seuil de sécurité (A)

#define JS1_X 13
#define JS1_Y 12
#define JS2_X 15
#define JS2_Y 25

/* Zone morte des joysticks analogiques (sur 4095) */
#define DEADZONE 200

```

```

/* -----
   Mode de contrôle
   0 = menu, 1 = PS5, 2 = analogique
   ----- */

int modeActuel = 0;

/* Valeurs de repos calibrées au démarrage du mode analogique */
int js1x_zero = 2048;
int js1y_zero = 2048;
int js2x_zero = 2048;
int js2y_zero = 2048;

/* -----
   Variables d'angles des servos
   ----- */

int angleBase    = 90;
int angleEpaule  = 90;
int angleCoude   = 90;
int anglePince   = 90;

/* Tension de référence du capteur de courant */
float tensionZero = 0;

/* -----
   Conversion angle → impulsion PWM
   ----- */

int angleToPulse(int ang) {
    return map(ang, 0, 180, SERVOMIN, SERVOMAX);
}

```

```

/* Déplacement sécurisé d'un servomoteur */

void moveServo(int servo, int angle) {
    angle = constrain(angle, 0, 180);
    pwm.setPWM(servo, 0, angleToPulse(angle));
}

/* -----
   Lecture d'un axe analogique avec zone morte
   Retourne une valeur entre -1.0 et +1.0
   ----- */

float lireAxe(int pin, int zero) {
    int val = analogRead(pin) - zero;
    if (abs(val) < DEADZONE) return 0.0;
    if (val > 0) return (float)val / (float)(4095 - zero);
    else      return (float)val / (float)zero;
}

/* Calibration des joysticks : moyenne sur 50 lectures au repos */

void calibrerJoysticks() {
    M5.Lcd.setCursor(0, 180);
    M5.Lcd.setTextColor(YELLOW);
    M5.Lcd.setTextSize(1);
    M5.Lcd.println("Calibration en cours...");
    M5.Lcd.println("Ne touchez pas les joysticks !");

    long s1x = 0, s1y = 0, s2x = 0, s2y = 0;
    for (int i = 0; i < 50; i++) {
        s1x += analogRead(JS1_X);
        s1y += analogRead(JS1_Y);
        s2x += analogRead(JS2_X);
    }
}

```

```

    s2y += analogRead(JS2_Y);
    delay(10);
}
js1x_zero = s1x / 50;
js1y_zero = s1y / 50;
js2x_zero = s2x / 50;
js2y_zero = s2y / 50;

Serial.printf("Zeros calibres: JS1X=%d JS1Y=%d JS2X=%d JS2Y=%d\n",
              js1x_zero, js1y_zero, js2x_zero, js2y_zero);
}

/* -----
   Affichage du menu de sélection
   ----- */
void afficherMenu() {
    M5.Lcd.clear();
    M5.Lcd.setTextSize(2);
    M5.Lcd.setTextColor(WHITE);

    M5.Lcd.setCursor(40, 10);
    M5.Lcd.setTextColor(YELLOW);
    M5.Lcd.println("=== BRAS ROBOTISE ===");
    M5.Lcd.setTextColor(WHITE);

    M5.Lcd.setCursor(20, 50);
    M5.Lcd.println("Choisissez le mode :");

    /* Bouton A = PS5 */
    M5.Lcd.fillRoundRect(10, 85, 140, 50, 8, BLUE);

```

```

M5.Lcd.setCursor(25, 100);

M5.Lcd.setTextColor(WHITE);

M5.Lcd.println(" [A] PS5");


/* Bouton B = Analogique */

M5.Lcd.fillRoundRect(165, 85, 145, 50, 8, RED);

M5.Lcd.setCursor(172, 100);

M5.Lcd.setTextColor(White);

M5.Lcd.println("[B] Analog");


M5.Lcd.setTextColor(LIGHTGREY);

M5.Lcd.setCursor(10, 155);

M5.Lcd.setTextSize(1);

M5.Lcd.println("Btn A = Manette Bluetooth PS5");

M5.Lcd.println("Btn B = 2 joysticks analogiques");

M5.Lcd.println("Btn C = Retour menu (en cours)");

}


/* -----
   Affichage écran mode PS5
   ----- */

void afficherEcranPS5() {

  M5.Lcd.clear();

  M5.Lcd.setTextSize(2);

  M5.Lcd.setTextColor(CYAN);

  M5.Lcd.setCursor(50, 5);

  M5.Lcd.println("MODE PS5");

  M5.Lcd.setTextColor(WHITE);

  M5.Lcd.setCursor(0, 35);

  M5.Lcd.setTextSize(1);

```

```

    M5.Lcd.println("JS gauche X -> Base");
    M5.Lcd.println("JS gauche Y -> Epaule");
    M5.Lcd.println("JS droit Y -> Coude");
    M5.Lcd.println("JS droit X -> Pince");
    M5.Lcd.println("Btn C      -> Menu");
}

/* -----
   Affichage écran mode Analogique
----- */

void afficherEcranAnalogique() {
    M5.Lcd.clear();
    M5.Lcd.setTextSize(2);
    M5.Lcd.setTextColor(GREEN);
    M5.Lcd.setCursor(20, 5);
    M5.Lcd.println("MODE ANALOGIQUE");
    M5.Lcd.setTextColor(WHITE);
    M5.Lcd.setCursor(0, 35);
    M5.Lcd.setTextSize(1);
    M5.Lcd.println("JS1 axe X -> Base");
    M5.Lcd.println("JS1 axe Y -> Epaule");
    M5.Lcd.println("JS2 axe Y -> Coude");
    M5.Lcd.println("JS2 axe X -> Pince");
    M5.Lcd.println("Btn C      -> Menu");
}

/* -----
   Initialisation
----- */

void setup() {
    M5.begin();

```

```

Serial.begin(115200);

Wire.begin(21, 22);

pwm.begin();
pwm.setPWMFreq(50);

/* Calibration du zéro du courant (ACS712) */
float somme = 0;
for (int i = 0; i < 185; i++) {
    somme += analogRead(PIN_COURANT_PINCE) * 3.3 / 4095.0;
    delay(5);
}
tensionZero = somme / 185.0;

/* Affichage du menu au démarrage */
afficherMenu();
}

/* -----
   Boucle principale
   ----- */

void loop() {
    M5.update();

    /* ---- MENU ---- */
    if (modeActuel == 0) {
        if (M5.BtnA.wasPressed()) {
            modeActuel = 1;
            angleBase = 90;

```



```

    angleEpaule = 90;
    angleCoude  = 90;
    anglePince  = 90;

    moveServo(SERVO_BASE,    angleBase);
    moveServo(SERVO_EPAULE, angleEpaule);
    moveServo(SERVO_COUDE,   angleCoude);
    moveServo(SERVO_PINCE,   anglePince);

    initBT_Controller();
    afficherEcranPS5();
}

if (M5.BtnB.wasPressed()) {
    modeActuel = 2;

    /* Remise à zéro des angles avant calibration */
    angleBase  = 90;
    angleEpaule = 90;
    angleCoude  = 90;
    anglePince  = 90;

    moveServo(SERVO_BASE,    angleBase);
    moveServo(SERVO_EPAULE, angleEpaule);
    moveServo(SERVO_COUDE,   angleCoude);
    moveServo(SERVO_PINCE,   anglePince);

    afficherEcranAnalogique();

    calibrerJoysticks(); // calibration SANS Bluetooth actif
}

return; // on reste dans le menu
}

/* ---- RETOUR AU MENU (Btn C) ---- */
if (M5.BtnC.wasPressed()) {
    modeActuel = 0;

    afficherMenu();
}

```

```

    return;
}

/* ---- MODE PS5 ---- */
if (modeActuel == 1) {
    controllerUpdate();

    angleBase    += var_axisX / 300;
    angleEpaule  -= var_axisY / 300;
    angleCoude   -= var_axisRY / 300;

    /* Lecture courant pince */
    float tensionActuelle = analogRead(PIN_COURANT_PINCE) * 3.3 / 4095.0;
    float courantPince = (tensionActuelle - tensionZero) / SENSIBILITE_ACS712;
    if (courantPince < 0) courantPince = 0;

    /* Contrôle sécurisé de la pince */
    int deltaPince = -var_axisRX / 300;
    if (deltaPince < 0) {
        anglePince += deltaPince;
    } else {
        if (courantPince < COURANT_MAX_PINCE) {
            anglePince += deltaPince;
        }
    }
}

/* Limites mécaniques */
angleBase    = constrain(angleBase, 0, 180);
angleEpaule  = constrain(angleEpaule, 5, 170);
angleCoude   = constrain(angleCoude, 10, 160);

```

```

anglePince = constrain(anglePince, 54, 180);

moveServo(SERVO_BASE, angleBase);
moveServo(SERVO_EPAULE, angleEpaule);
moveServo(SERVO_COUDE, angleCoude);
moveServo(SERVO_PINCE, anglePince);

/* Affichage périodique */
static unsigned long tPS5 = 0;
if (millis() - tPS5 > 200) {
    tPS5 = millis();
    M5.Lcd.setCursor(0, 80);
    M5.Lcd.fillRect(0, 80, 320, 160, BLACK);
    M5.Lcd.setTextSize(2);
    M5.Lcd.setTextColor(WHITE);
    M5.Lcd.printf("Base   : %d\n", angleBase);
    M5.Lcd.printf("Epaule : %d\n", angleEpaule);
    M5.Lcd.printf("Coude  : %d\n", angleCoude);
    M5.Lcd.printf("Pince  : %d\n", anglePince);
    M5.Lcd.setTextColor(courantPince > COURANT_MAX_PINCE * 0.8 ? RED :
GREEN);
    M5.Lcd.printf("Courant: %.2f A\n", courantPince);
    M5.Lcd.setTextColor(WHITE);
}
}

/* ---- MODE ANALOGIQUE ---- */
else if (modeActuel == 2) {
    /* Lecture des axes avec zéros calibrés */
    float js1x = lireAxe(JS1_X, js1x_zero);
    float js1y = lireAxe(JS1_Y, js1y_zero);

```

```

float js2x = lireAxe(JS2_X, js2x_zero);
float js2y = lireAxe(JS2_Y, js2y_zero);

/* Vitesse de déplacement en degrés/cycle */
float vitesse = 1.5;

angleBase   += js1x * vitesse;
angleEpaule -= js1y * vitesse;
angleCoude  -= js2y * vitesse;

/* Lecture courant pince */
float tensionActuelle = analogRead(PIN_COURANT_PINCE) * 3.3 / 4095.0;
float courantPince = (tensionActuelle - tensionZero) / SENSIBILITE_ACS712;
if (courantPince < 0) courantPince = 0;

/* Contrôle sécurisé de la pince */
float deltaPince = js2x * vitesse;
if (deltaPince < 0) {
    anglePince += deltaPince;
} else {
    if (courantPince < COURANT_MAX_PINCE) {
        anglePince += deltaPince;
    }
}

/* Limites mécaniques */
angleBase   = constrain(angleBase, 0, 180);
angleEpaule = constrain(angleEpaule, 5, 170);
angleCoude  = constrain(angleCoude, 10, 160);
anglePince  = constrain(anglePince, 54, 180);

```

```

moveServo(SERVO_BASE,    angleBase);
moveServo(SERVO_EPAULE,  angleEpaule);
moveServo(SERVO_COUDE,   angleCoude);
moveServo(SERVO_PINCE,   anglePince);

/* Affichage périodique */
static unsigned long tAna = 0;
if (millis() - tAna > 200) {
    tAna = millis();
    M5.Lcd.setCursor(0, 80);
    M5.Lcd.fillRect(0, 80, 320, 160, BLACK);
    M5.Lcd.setTextSize(2);
    M5.Lcd.setTextColor(WHITE);
    M5.Lcd.printf("Base    : %d\n", angleBase);
    M5.Lcd.printf("Epaule  : %d\n", angleEpaule);
    M5.Lcd.printf("Coude   : %d\n", angleCoude);
    M5.Lcd.printf("Pince   : %d\n", anglePince);
    M5.Lcd.setTextColor(courantPince > COURANT_MAX_PINCE * 0.8 ? RED :
GREEN);
    M5.Lcd.printf("Courant: %.2f A\n", courantPince);
    M5.Lcd.setTextColor(WHITE);
}
}

delay(20);
}

```